# A plotting cheat sheet

JACS plotting session · Jodrell Bank · 12 November 2026

📈 **Make plots that tell the truth, and look good.** Everything from the JACS plotting session (Jodrell Bank, 12 November 2026) on one page. Skim the rules, copy the code, come back when you need it.

**▶️ Run the code**

`jacs_solutions.ipynb` : every plot, bad to good.

**✍️ Try it yourself**

`jacs_participant.ipynb` : the same exercises, with blanks to fill in.

**🧰 Set it up**

Section 1: environment and kernel in 5 steps.


⚡ At a glance: 12 rules
🚀 1. Houston, we need a kernel
🥷 2. Choose your fighter
📊 3. Bad plot, good plot
💅 4. Time for a makeover
🕵️ 5. Seems legit: a cautionary tale
🖱️ 6. Bonus: interactive plots
📎 Appendix: the libraries and the options you'll reach for
🙏 Data credits and references


## ⚡ At a glance: 12 rules

**🎯 Choose and label**

**1.** Choose the plot from the question, not from the data. (Section 2)

**2.** Label every axis with units. Label the colourbar, and say so if its scale is nonlinear. (Section 3)

**3.** Match the colormap to the data: sequential for ordered data, diverging centred on zero for signed data, qualitative for groups. Never `jet` or rainbow. (4.1)

**🔬 Be honest about the data**

**4.** A line through the points is not a test of the fit. Add a residual panel and quote chi2/dof. (3.2)

**5.** Don't let the binning choose your story. Use an ECDF, or a rule-based bin width such as `bins="fd"` . (3.4)

**7.** A corner plot cannot show convergence. Check traces, per-chain KDEs and split R-hat first. (3.6)

**🖼️ Show it right**

**6.** For pairwise quantities use a heatmap: diverging colormap centred on zero, sorted, annotated, each pair once. (3.5)

**8.** Images and maps need the right orientation, stretch and coordinates: `origin="lower"` , asinh or log scale, WCS axes, and a statement of what you clipped. (3.7 and 3.8)

**9.** Colour is never the only cue. Add linestyles, markers and hatching, and test in grayscale. (4.1)

**🛠️ Work like a pro**

**10.** Decide the look once, in an `.mplstyle` file, and never hard-code colours or fonts inside plotting functions. (4.4)

**11.** Check every AI-made plot: colour limits, number of points, whose data it is, and a known-answer test. (Section 5)

**12.** Static plots are for papers. Interactive plots are for exploring and sharing. (Section 6)

### 🔁 Bad to good, plot by plot

| Plot | The common crime | The fix | Key code |
|---|---|---|---|
| **1D power spectrum** | `plot(cl)` on linear axes, no labels, monopole and dipole included | Plot D_ell = ell(ell+1)C_ell/2π on log axes from ell = 2, with labels and units. Then bin, add errors, mark the beam. | `ax.loglog(ell[2:], D[2:])` |
| **A fit that looks perfect** | Data and model on log-log axes only | Residual panel, chi2/dof | `gridspec_kw={"height_ratios": [3, 1.5]}` |
| **2D power spectrum** | Raw FFT on `jet` | Remove the mean, taper, centre, log colour scale, ell axes | `LogNorm(vmin=P2d.max() * 1e-6)` |
| **Histogram** | One arbitrary bin count | ECDF next to a rule-based histogram | `bins="fd"` |
| **Heatmap of pairwise differences** | Default colormap, random order, every pair twice | Diverging map centred on zero, sorted, annotated, each pair once | `cmap="RdBu_r", vmin=-vmax, vmax=vmax` |
| **Convergence and corner plots** | Trusting the pooled corner plot | Trace plots, per-chain KDEs, split R-hat | `split_rhat(chains)` |
| **Image (Horsehead)** | Upside down, linear stretch, `jet`, pixel axes | `origin="lower"`, asinh stretch, WCS axes, greyscale | `simple_norm(image, stretch="asinh", percent=99.5)` |
| **Sky map** | Linear scale, no units or title | Sequential map, clipped or log scale, units on the colourbar | `hp.mollview(sky, norm="log")` |
| **Forest plot** | Bar chart with rainbow colours | Horizontal error bars, grouped, with units | `ax.errorbar(H0, y, xerr=H0_err)` |

---

## 🚀 1. Houston, we need a kernel

To run the notebooks yourself:

1. Download or clone the repo.
2. Install Miniconda.
3. In a terminal, from the repo folder:

```
conda env create -f environment.yml
conda activate jacs-plotting
python -m ipykernel install --user --name jacs-plotting --display-name "JACS plotting"
```

4. Open the notebook in Jupyter (`jupyter lab`) or VS Code, and pick the **JACS plotting** kernel. In VS Code the kernel picker is at the top right of the notebook.
5. Run the first cell. "All good" means you are ready. If it lists something missing, install it with `pip install <package>`, or check that the `data/` folder is next to the notebook.

> 🩹 **If something fails:** a new kernel sometimes only appears in the list after you restart Jupyter or VS Code. On Windows, if `healpy` fails to install, use WSL. In VS Code, links in notebook cells open with Ctrl-click (Windows and Linux) or Cmd-click (macOS).

---

## 🥷 2. Choose your fighter

Before you make a nice plot, choose the right plot. Start from the question you are asking:

| Question | Plot |
|---|---|
| How is it distributed? | Histogram or KDE; box or violin plots to compare groups |
| How are two things related? | Scatter with a fit and a residual panel |
| How do many pairs compare? | Heatmap |
| How do several estimates with uncertainties compare? | Forest plot |
| How does it change with time or frequency? | Line plot with an uncertainty band |
| What does a 2D field look like? | Image or sky map with a colourbar and a sensible stretch |
| What does a joint posterior look like? | Corner plot (after checking convergence) |

📚 **Useful resources**

- From Data to Viz: a decision tree for choosing a chart from the shape of your data.
- Cleveland, W. S. and McGill, R., 1984, Journal of the American Statistical Association, 79, 531, "Graphical perception: theory, experimentation, and application to the development of graphical methods" (why some encodings, such as position along a common scale, are read more accurately than others, such as colour or area).

## 📊 3. Bad plot, good plot

✅ **Better-plot checklist**

- [ ] Axis labels with units
- [ ] Colourbar with a label (and say if the scale is nonlinear)
- [ ] A sensible stretch and limits; state any clipping
- [ ] Correct orientation and coordinates
- [ ] Colourblind-safe colours; never rainbow
- [ ] Diverging colormap centred on zero for signed data
- [ ] Legend does not cover the data
- [ ] Size and fonts matched to where the figure will be seen

Each plot below follows the same pattern: ❌ **bad**, 🙂 **better**, 🌟 **best**.

### 3.1 The 1D power spectrum

The data: the Haslam 408 MHz all-sky map (brightness temperature in K), with `cl = hp.anafast(m, lmax=700)`.

❌ **Bad:** `plt.plot(cl)`. The spectrum falls steeply, so on linear axes everything above ell of about 20 is squashed flat. There are no labels or units, and the monopole and dipole are included.

🙂 **Better:** plot D_ell = ell(ell+1)C_ell/2π on log axes, start at ell = 2, and label the axes.

```
ell = np.arange(len(cl))
D = ell * (ell + 1) * cl / (2 * np.pi)

fig, ax = plt.subplots(figsize=(6, 4))
ax.loglog(ell[2:], D[2:])                       # log-log; drop the monopole and dipole
ax.set_xlabel(r"Multipole $\ell$")
ax.set_ylabel(r"$D_\ell\ [\mathrm{K}^2]$")
plt.show()
```

🌟 **Best:** bin in log-spaced bins, add error bars, and shade the part of the spectrum that the beam smooths away.

- The sample-variance error is `D_b * np.sqrt(2 / n_modes)`, where `n_modes` is the number of modes in each bin. It is only a rough guide, because foregrounds are not Gaussian.
- The 56 arcmin beam halves the power at ell of about 120: `ell_beam = np.sqrt(np.log(2)) / (np.radians(56 / 60) / np.sqrt(8 * np.log(2)))`. The drop beyond it is the telescope, not the sky.

## 3.2 A fit that looks perfect

🎭 A power law fitted to 20 points with 5% errors can look perfect on log-log axes, because a deviation of up to about 30% is tiny next to four decades in y. **A line passing through the points is not a test of the fit: look at the residuals.**

```
fig, (ax1, ax2) = plt.subplots(2, 1, sharex=True, figsize=(5, 5),
                               gridspec_kw={"height_ratios": [3, 1.5]})
ax1.errorbar(x, y, yerr=sigma, fmt="o", ms=4, label="data")
ax1.plot(x, model, "k-", lw=1, label=rf"power law, index $= {index:.2f}$")
ax1.set_yscale("log")
ax1.legend()

ax2.axhline(0, color="k", lw=1)
ax2.plot(x, resid, "o", ms=4)                   # resid = (data - model) / sigma
ax2.set_xscale("log")
ax2.set_ylabel(r"(data $-$ model)$/\sigma$")
ax2.text(0.03, 0.8, rf"$\chi^2/\mathrm{{dof}} = {chi2_dof:.1f}$", transform=ax2.transAxes)
```

Here chi2/dof is about 7, so the power law is rejected, even though the picture says it fits. The index you quote from the wrong model has little meaning.

## 3.3 The 2D power spectrum

A 20 by 20 degree flat-sky patch of the Haslam map, cut out with `hp.gnomview(..., return_projected_map=True, no_plot=True)`.

❌ **Bad:** `plt.imshow(P2d_raw, cmap="jet")` on the raw FFT output. The power sits in the corners and is invisible on a linear scale.

🙂 **Better:** remove the mean, taper the edges, shift the zero frequency to the centre, use ell axes and a log colour scale, and mark the beam scale.

```
window = np.outer(np.hanning(n), np.hanning(n))                      # taper the edges
F = np.fft.fftshift(np.fft.fft2((patch - patch.mean()) * window))    # mean removed, zero frequency centred
P2d = np.abs(F) ** 2 * dtheta**2 / (n**2 * np.mean(window**2))       # K^2 sr, same units as C_ell
ell_xy = 2 * np.pi * np.fft.fftshift(np.fft.fftfreq(n, d=dtheta))

im = ax.pcolormesh(ell_xy, ell_xy, P2d, norm=LogNorm(vmin=P2d.max() * 1e-6), cmap="viridis")   # top 6 decades
ax.add_patch(plt.Circle((0, 0), ell_beam, fill=False, color="w", ls="--"))
ax.set_aspect("equal")
```

A single map gives only the two angular directions. A cylindrical P(k_perp, k_parallel), as in 21cm work, needs many frequency channels.

## 3.4 Histogram binning

🎭 The same 300 points shown with 8, 25 and 80 bins tell three different stories: coarse bins merge two populations into one skewed peak, medium bins show two, and fine bins show mostly noise. **If a feature appears or disappears when the binning changes, the plot is not evidence for it.**

```
xs = np.sort(x_h)
edges = np.histogram_bin_edges(x_h, bins="fd")    # Freedman-Diaconis bin width

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(9, 3.5))
ax1.step(xs, np.arange(1, len(xs) + 1) / len(xs), where="post")   # ECDF: no binning at all
ax2.hist(x_h, bins=edges)
ax2.set_title(f"{len(edges) - 1} bins (Freedman-Diaconis)")
```

## 3.5 Heatmaps for pairwise quantities

A heatmap is the right tool when you want to compare every pair: here the difference in H0 between six probes (synthetic values).

❌ **Bad:** `plt.imshow(D[np.ix_(order, order)])`. The default colormap does not mark zero, the order is random, and every pair appears twice.

🙂 **Better:** a diverging colormap centred on zero, labels, and each pair once.

🌟 **Best:** sort by value so that groups show up as blocks, and annotate the cells, because heatmaps are weak at exact numbers. Outline the cells whose difference is smaller than the combined 1 sigma uncertainty.

```
srt = np.argsort(h0)
Ds, Ss = D[np.ix_(srt, srt)], S[np.ix_(srt, srt)]              # D = h0[:, None] - h0[None, :]
lower = np.tril(np.ones((6, 6), bool), k=-1)                    # each pair once
vmax = np.abs(D).max()

im = ax.imshow(np.where(lower, Ds, np.nan), cmap="RdBu_r", vmin=-vmax, vmax=vmax)
for i, j in zip(*np.nonzero(lower)):
    ax.text(j, i, f"{Ds[i, j]:+.1f} ± {Ss[i, j]:.1f}", ha="center", va="center", fontsize=7)
    if abs(Ds[i, j]) < Ss[i, j]:                                # consistent within combined 1 sigma
        ax.add_patch(plt.Rectangle((j - 0.5, i - 0.5), 1, 1, fill=False, ec="k", lw=1))
```

## 3.6 Convergence, and corner plots

🚨 **A corner plot cannot tell you whether your chains converged.** An early-stopped run can give a clean, pooled corner plot with tidy medians and errors.

To check:

1. **Trace plots,** with the burn-in marked and the sample axis on a log scale so the early steps are visible.
2. **Per-chain KDEs:** overlay one distribution per chain, and check that they agree.
3. **Split R-hat:** it should be close to 1.00 (below about 1.01). It is noisy on very short chains, so treat it as a warning sign rather than a precise number.

```
def split_rhat(x):
    """Split R-hat for one parameter; x has shape (chain, sample)."""
    n = x.shape[1] // 2
    x = np.concatenate([x[:, :n], x[:, n:2 * n]])       # cut every chain into two halves
    W = x.var(axis=1, ddof=1).mean()                     # within-chain variance
    B = n * x.mean(axis=1).var(ddof=1)                   # between-chain variance
    return np.sqrt(((n - 1) / n * W + B / n) / W)
```

Only then make the final corner plot, with the burn-in removed. Ideally use independent chains from dispersed starting points. A scatter-plus-histogram grid made with a `for` loop is not a corner plot: use the `corner` library.

```
fig = corner.corner(samples, labels=names, truths=truth,
                    levels=(1 - np.exp(-0.5), 1 - np.exp(-2)),   # 1 and 2 sigma in 2D
                    quantiles=[0.16, 0.5, 0.84], show_titles=True, title_fmt=".3f",
                    plot_datapoints=False, fill_contours=True)
```

🧱 **When a parameter piles up against a prior bound,** median plus or minus error is the wrong summary. Show the prior bounds, state the contour levels, and quote an upper limit.

```
ul = np.percentile(theta3, 95)
fig = corner.corner(samples_b, labels=names_b, levels=(0.68, 0.95), show_titles=True,
                    range=[0.999, 0.999, (-0.05, 1.05), 0.999])
corner.overplot_lines(fig, [None, None, 0, None], color="k", ls=":")   # prior bounds
corner.overplot_lines(fig, [None, None, 1, None], color="k", ls=":")
```

## 3.7 Images: the Horsehead Nebula

❌ **Bad:** `plt.imshow(image, cmap="jet")` . FITS puts pixel (0, 0) at the bottom left but `imshow` puts it at the top left, so the image is upside down. The axes are pixel numbers although the header has a WCS, a linear stretch squeezes the faint nebula into a few grey levels, and `jet` adds structure that is not there.

🙂 **Better:** `origin="lower"` , greyscale, labelled axes and a colourbar.

🌟 **Best:** an asinh stretch and sky coordinates. Always state the clipping and say the colourbar is nonlinear.

```
norm = simple_norm(image, stretch="asinh", percent=99.5)   # clips the brightest 0.5%, so the stars saturate

fig = plt.figure(figsize=(6, 6))
ax = fig.add_subplot(projection=wcs)                       # wcs = WCS(header)
im = ax.imshow(image, origin="lower", cmap="gray", norm=norm)
ax.coords.grid(color="white", alpha=0.5, ls=":")
ax.set_xlabel("RA (ICRS)")
ax.set_ylabel("Dec (ICRS)")
fig.colorbar(im, ax=ax, label="Plate density (arbitrary units, asinh stretch)", shrink=0.8)
```

## 3.8 Sky maps

The Galactic plane (up to about 4000 K) is hundreds of times brighter than the faint sky (10 to 50 K), so a linear scale either saturates the plane or hides everything else, including the North Polar Spur and the other loops.

❌ **Bad:** `hp.mollview(sky)` : no title, no units, no coordinate system.

🙂 **Better:** a sequential colormap, a clipped linear scale (and say that you clipped it), units and a title.

```
lo, hi = np.percentile(sky, [1, 95])              # the brightest 5% of pixels (the plane) saturate
hp.mollview(sky, cmap="inferno", min=lo, max=hi, unit="K",
            title=f"Haslam 408 MHz, Galactic coordinates, linear scale clipped at {hi:.0f} K")
```

🌟 **Best:** a log colour scale, with the colourbar ticks relabelled in real units. The colourbar is no longer linear, so it must say so.

```
hp.mollview(sky, norm="log", cmap="inferno", cbar=False,
            title="Haslam 408 MHz, Galactic coordinates, log colour scale")
ticks_K = [15, 30, 100, 300, 1000, 3000]
cb = plt.colorbar(plt.gca().get_images()[0], orientation="horizontal", shrink=0.6, pad=0.05, ticks=ticks_K)
cb.set_ticklabels([f"{t:g}" for t in ticks_K])
cb.set_label("Brightness temperature [K], log scale")
```

Nothing was removed from the map: only the colour scale changed.

### 3.9 Forest plots

❌ **Bad:** a bar chart with rainbow colours and rotated labels, which makes a comparison of estimates with uncertainties very hard to read.

🙂 **Better:** horizontal error bars, grouped, with a shaded band and units.

```
y = np.arange(len(H0))[::-1]
ax.errorbar(H0[early], y[early], xerr=H0_err[early], fmt="o", color="C0", label="Early Universe")
ax.errorbar(H0[~early], y[~early], xerr=H0_err[~early], fmt="s", color="C1", label="Late Universe")
ax.set_yticks(y)
ax.set_yticklabels(labels)
ax.set_xlabel(r"$H_0\ [\mathrm{km\,s^{-1}\,Mpc^{-1}}]$")
```

(The H0 values are approximate and only for illustration.)

---

## 💅 4. Time for a makeover

💡 A good-looking plot is a set of decisions. Make them once, then let a file apply them everywhere.

### 4.1 How to pick a colour scheme

The palette follows the type of data:

| Your data | Palette type | Try | Watch out |
|---|---|---|---|
| Continuous and ordered (temperature, density) | Sequential | `viridis`, `cividis`, `magma` | Never rainbow or `jet`: it invents edges |
| Signed, with a meaningful zero (differences, residuals) | Diverging, centred on zero | `RdBu_r`, `coolwarm` with symmetric limits | Centre on the zero, not on the middle of the data |
| A few distinct groups | Qualitative | Okabe-Ito, `tab10`, `Set2` | About 6 to 8 colours at most |
| Many lines | Sample a sequential map evenly, or use small multiples | `plt.cm.viridis(np.linspace(0, 1, N))` | If readers must look up a legend, label the lines directly |

**Is it colourblind-friendly?**

- Run a simulator on your figure, such as the Colorblindly browser extension.
- Try the grayscale test: if the lines are indistinguishable in grayscale, colour is carrying too much.

**Colour often isn't enough: add texture.** Different linestyles and markers, and a hatched region instead of a tint, survive grayscale printing and colour-vision deficiencies.

```
ax.axvspan(0, 590, facecolor="none", edgecolor="0.5", hatch="///")   # hatched region
ax.plot(x, y1, "-",  marker="o")                                      # different linestyles and markers
ax.plot(x, y2, "--", marker="s")
ax.plot(x, y3, ":",  marker="^")
plt.rcParams["hatch.linewidth"] = 0.6
```

The Okabe-Ito colours are a good colourblind-safe set: `#0072B2`, `#E69F00`, `#009E73`, `#D55E00`, `#CC79A7`, `#56B4E9`.

**Useful resources**

- ColorBrewer: palettes for maps and plots, with a colourblind-safe filter.
- Matplotlib colormaps reference
- Wong, B., 2011, Nature Methods, 8, 441, "Points of view: Color blindness".
- Crameri, F., Shephard, G. E. and Heron, P. J., 2020, Nature Communications, 11, 5444, "The misuse of colour in science communication".

## 4.2 Customising your colour palette

### Make your own

- Gradient generator: build a gradient by picking colours, then copy the hex codes.
- Coolors: generate and tweak palettes, and export the hex codes.

```
from matplotlib.colors import LinearSegmentedColormap
hexes = ["#03045E", "#0077B6", "#00B4D8", "#90E0EF", "#CAF0F8"]   # dark to light, from the gradient generator
cmap = LinearSegmentedColormap.from_list("mine", hexes)
```

A gradient is for continuous data. For separate lines pick distinct colours instead: neighbouring samples of a gradient look alike, and nobody can match a line to "the fourth shade of pink". Sample a gradient only when the lines themselves are ordered (redshift bins, frequencies, time).

A custom palette is not automatically perceptually uniform, so run the grayscale test: the lightness should rise or fall steadily. A pretty palette can fail it. For example `#1B2021, #30343F, #89023E, #EA638C, #FFD9DA` looks good, but its second and third stops have almost the same lightness, so the mid-range nearly disappears in grayscale.

### Ready-made

- Built in: `tab10`, `tab20`, `Set2`, `Dark2`, `cividis`. `plt.colormaps()` lists them all.
- CMasher: colormaps made for astronomy.
- cmocean: perceptually uniform colormaps.
- colorcet: perceptually uniform colormaps by Peter Kovesi.
- `cmcrameri`: Fabio Crameri's scientific colour maps (`pip install cmcrameri`).
- seaborn: `sns.color_palette("colorblind")`.

**Worked example: a palette with roles.** Pick colours that mean something, and give each one a job. Here gold is always the result you want the eye to find, and everything else is cooler or darker.

| Role | Hex | Use it for |
|---|---|---|
| 🟨 Hero (burnished gold) | `D99A00` | Your headline data or result |
| 🟦 Mid blue | `3E6FB0` | Other datasets |
| 🟥 Ember red | `C8452D` | Predictions |
| 🟩 Deep teal | `2A9D8F` | A second model |
| 🟪 Muted violet | `7B5EA7` | A third group |
| ⬛ Ink navy | `1F2A44` | Fits, text, axes |

For maps, use a gradient from midnight through violet and ember to gold ( `0B132B` , `5B3F8C` , `C8452D` , `FFD166` ). For signed data use blue, near-white, ember, centred on zero, and keep the hero colour out of it. Gold on white is low contrast, so give gold markers a dark edge.

**Put a palette in the style file**

- Discrete colours go straight into `axes.prop_cycle` , written without the `#` .
- A style file cannot define a gradient, only name one. Register the colormap in code first, then name it in the file:

```
import matplotlib as mpl
from matplotlib.colors import LinearSegmentedColormap

hexes = ["#0B132B", "#5B3F8C", "#C8452D", "#FFD166"]
mpl.colormaps.register(LinearSegmentedColormap.from_list("ring", hexes))
# then in the .mplstyle file:  image.cmap: ring
# register BEFORE calling plt.style.use
```

Run the grayscale test and a colour-vision simulator on any palette before you commit to it.

## 4.3 Typesetting text in plots

**Text on top of a plot**

```
ax.text(0.05, 0.9, "Case I", transform=ax.transAxes)              # axes coordinates: stays put
ax.annotate("peak", xy=(x0, y0), xytext=(x1, y1), arrowprops=dict(arrowstyle="->"))
ax.text(..., bbox=dict(facecolor="white", alpha=0.8, edgecolor="none"))   # readable over data
```

Label lines directly instead of using a legend whenever you can, for example at the right edge of each line.

**Fonts and sizes (rcParams).** Set them once, not in every call:

```
plt.rcParams.update({
    "font.size": 12,
    "axes.labelsize": 14,
    "xtick.labelsize": 11,
    "ytick.labelsize": 11,
})
```

- Other keys: `font.family` , `legend.fontsize` .
- Design at the final size: text should be about 8 to 10 pt in a journal column (roughly 3.4 inches wide), and 14 pt or more on slides.

**Maths and LaTeX**

- `mathtext` needs no install: `r"$\ell(\ell+1)C_\ell/2\pi$"`. Set `mathtext.fontset: stix` to match a Times-like text font. STIX is a font family designed for scientific publishing.
- Full LaTeX (`text.usetex: True`) needs LaTeX installed and is slower. Guard it: `if shutil.which("latex"): ...`.

> 📚 **Useful resources**
>
> - Text in Matplotlib
> - Writing mathematical expressions (mathtext)
> - Text rendering with LaTeX

## 4.4 A uniform style across the project: the mplstyle file

An `.mplstyle` file is a plain-text list of matplotlib settings (rcParams), one `key: value` per line. `#` starts a comment, so write hex colours without it. `plt.style.use("file.mplstyle")` applies all of them at once, so every plot follows the same decisions.

> 📏 **Two rules**
>
> - **Don't hard-code colours or fonts inside plotting functions.** If you do, the style file cannot reach them. The function says *what* to draw, and the style decides *how* it looks. Pass a colour in only when it carries meaning.
> - **Keep a paper style and a talk style.** A small `talk.mplstyle` with bigger fonts layered on top: `plt.style.use(["paper.mplstyle", "talk.mplstyle"])`.

### A starter file

```
# jacs.mplstyle
figure.figsize: 6.4, 4.0
savefig.dpi: 300
savefig.bbox: tight
font.size: 11
axes.labelsize: 12
legend.fontsize: 10
lines.linewidth: 1.8
xtick.direction: in
ytick.direction: in
xtick.top: True
ytick.right: True
xtick.minor.visible: True
ytick.minor.visible: True
legend.frameon: False
axes.prop_cycle: cycler('color', ['0072B2', 'E69F00', '009E73', 'D55E00', 'CC79A7', '56B4E9'])
```

### Use it

```
plt.style.use("jacs.mplstyle")                # whole notebook
with plt.style.context("jacs.mplstyle"):      # one figure
    ...
plt.style.use(["default", "jacs.mplstyle"])   # reset first, then apply
```

✅ **What a style file can hold:** any rcParam. Figure size and dpi; fonts; axes (label sizes, grid, spines, `axes.prop_cycle`); ticks (direction, top and right, minor); lines and markers; legend; images (`image.cmap`, `image.origin`); saving (`savefig.dpi`, `savefig.bbox`); hatching (`hatch.linewidth`). `plt.rcParams.keys()` lists every setting, and `plt.style.available` lists the built-in styles.

🚫 **What it cannot hold:** anything that is not an rcParam (annotations, colourbar labels, norms and stretches, per-plot choices), logic such as loops or conditions, and a few global settings such as the backend.

**Rules and gotchas**

- Later styles win, and anything you set in code afterwards overrides the file.
- A style only changes the keys in its file. A font or setting you changed earlier in the notebook stays active, so start from `["default", "file.mplstyle"]`.
- Values are not quoted, with one exception: inside `cycler(...)` the colour strings are quoted.
- To give each line its own colour, linestyle and marker, combine cyclers: `axes.prop_cycle: cycler(color=['0072B2', 'E69F00', '009E73']) + cycler(linestyle=['-', '--', ':']) + cycler(marker=['o', 's', '^'])`.
- Copy your style file into `mpl.get_configdir() + "/stylelib/"` and it loads by name from any folder.
- `text.usetex: True` needs LaTeX installed. In Jupyter, call `plt.style.use(...)` near the top, after the imports.

📚 **Useful resources**

- Customizing Matplotlib with style sheets and rcParams
- SciencePlots (`pip install SciencePlots`): ready-made journal styles. It uses LaTeX by default; add the `no-latex` style if you do not have it installed.

---

## 🕵️ 5. Seems legit: a cautionary tale

🔎 The code ran. The plot looks fine. It can still be wrong.

**The story:** I asked an AI for a subplot grid with one colourbar shared by all panels. I got a figure with one colourbar. The panels actually had different ranges, so the colourbar was only true for one of them. I did not notice for a long time.

⏱️ **The 10-second check:**

```
[im.get_clim() for ax in axes for im in ax.images]   # a shared colourbar needs one range
```

🔧 **The fix:** compute one `vmin` and `vmax` across all panels, pass them to every `imshow`, then call `fig.colorbar(im, ax=axes)`.

**Other ways an AI-made plot can look right and be wrong**

- It used made-up data when you asked for your real data.

- It dropped NaNs or masked points to "fix" an error, or a log axis hid zeros and negatives (a symlog axis shows them all).
- Axis limits that hide an outlier, or smoothing and interpolation you did not ask for.
- Guessed units or labels, legend entries in the wrong order, a flipped image origin.
- A function argument that exists but means something different.

📝 **Before you trust an AI-made plot:**

- [ ] Did I ask for the check as well as the plot?
- [ ] Did I read the code, not just look at the picture?
- [ ] Are the colour limits, axis limits and scales what I intended?
- [ ] Does the number of points match my data?
- [ ] Is the data mine, and not simulated?
- [ ] Did I test with an input where I know the answer?
- [ ] Do a few values match the raw numbers?
- [ ] Can I explain every line to a reviewer?

**The plot isn't done when it renders. It's done when you've checked it.**

---

## 🖱️ 6. Bonus: interactive plots

💡 Static plots are for papers. Interactive plots are for exploring your data and sharing it. You do not have to choose: explore interactively, then make the static version.

| Tool | What you get | Best for |
|---|---|---|
| **Plotly** | Hover, zoom, pan, click-to-hide legend entries; exports to a standalone HTML file | Exploring scatter, line and image data; sharing a figure by file or link |
| **ipympl** ( `%matplotlib widget` ) | Your existing matplotlib figure, with pan and zoom in the notebook | Keeping your matplotlib code and style file |
| **ipywidgets** | Sliders and dropdowns that re-draw a figure | Seeing what a parameter does (a stretch, a smoothing width, a bin count) |
| **Bokeh, Altair, HoloViews** | Larger interactive dashboards and linked views | When you outgrow the three above |

**Install (optional; not in the session environment, and ipywidgets may already be there):**

```
conda activate jacs-plotting
pip install plotly nbformat ipywidgets ipympl
```

In VS Code, `nbformat` is needed for Plotly figures to render. After installing, restart the kernel.

### Plotly: hover and zoom

```
import plotly.express as px
fig = px.scatter(df, x="x", y="y", color="group", hover_data=["name"])   # df is a pandas DataFrame
fig.update_layout(template="simple_white")
fig.show()
fig.write_html("figure.html")   # a standalone file you can send to anyone

fig = px.imshow(image, origin="lower", color_continuous_scale="viridis")   # an image you can zoom into
fig.show()
```

### ipywidgets: a slider that re-draws

```
import numpy as np, matplotlib.pyplot as plt
from ipywidgets import interact, FloatSlider

def show(s=300):
    plt.imshow(np.arcsinh(image / s), origin="lower")
    plt.show()

interact(show, s=FloatSlider(min=10, max=3000, step=10, value=300))
```

### ipympl: pan and zoom with plain matplotlib

```
%matplotlib widget
import matplotlib.pyplot as plt
fig, ax = plt.subplots()
ax.plot(x, y)
```

Put `%matplotlib inline` back when you want static figures again.

⚠️ **Gotchas**

- Interactive figures are not for a journal PDF. Make the static version for the paper, with your style file.
- Plotly figures use their own styling, so the mplstyle file does not apply to them.
- A notebook with interactive outputs saved can become very large. Clear outputs before committing.
- Interactive output does not show on a printed or exported PDF of a notebook, and on GitHub it may show blank. Export to HTML to share.
- Large images or maps (for example a full-resolution healpy map) are too heavy to send to the browser. Downsample first.
- Sliders only update while the kernel is running. They stop working in an exported file.

📚 **Useful resources:** Plotly for Python, ipywidgets, ipympl, Bokeh, Altair.

---

## 📎 Appendix: the libraries and the options you'll reach for

▶️ Runnable demos of every option below (linestyles, markers, hatches, colormaps, colour scaling, layout, and a quick tour of each library) are in the appendix of `jacs_solutions.ipynb`. Copy what you need from there.

## A. The libraries in this session

**NumPy** (`import numpy as np`): arrays and maths. Docs

- `np.linspace(a, b, n)`, `np.logspace`, `np.arange`, `np.meshgrid`
- `rng = np.random.default_rng(seed)` then `rng.normal(...)` for reproducible random numbers
- `np.arcsinh`, `np.log10`, `np.percentile`, `np.histogram`
- `np.polyfit(x, y, deg)` and `np.polyval(p, x)` for a quick fit
- `np.fft.fft2` and `np.fft.fftshift` for 2D spectra

**Matplotlib** (`import matplotlib.pyplot as plt`): the plotting workhorse. Docs

- `fig, ax = plt.subplots(figsize=(6, 4))`
- `ax.plot`, `ax.scatter`, `ax.errorbar`, `ax.hist`, `ax.imshow`, `ax.fill_between`, `ax.axvline`, `ax.axhline`, `ax.axvspan`
- `ax.set(xlabel=..., ylabel=..., xlim=..., title=...)`, `ax.set_xscale("log")`
- `ax.text`, `ax.annotate`, `ax.legend`
- `fig.colorbar(im, ax=ax, label="...")`
- `fig.savefig("fig.pdf", dpi=300, bbox_inches="tight")`
- `plt.style.use(...)`, `plt.rcParams[...]`

**healpy** (`import healpy as hp`): maps on the sphere. Docs

- `hp.read_map("file.fits")`
- `hp.mollview(m, title=..., unit=..., min=..., max=..., cmap=..., norm="hist")` for a full-sky map, with `hp.graticule()` to add a grid
- `hp.gnomview(m, rot=(lon, lat), xsize=..., reso=...)` for a flat patch of sky; `return_projected_map=True, no_plot=True` returns it as an array
- `hp.anafast(m, lmax=...)` for the angular power spectrum `C_ell`
- `hp.ud_grade(m, nside)` to change resolution, `hp.nside2npix`, `hp.pix2ang`

**Astropy** (`from astropy.io import fits`, `from astropy.wcs import WCS`): FITS files and sky coordinates. Docs

- `fits.open("file.fits")`, `hdul[0].data`, `hdul[0].header`
- `wcs = WCS(header)` then `fig.add_subplot(projection=wcs)` for RA/Dec axes; `ax.coords.grid(...)`
- `from astropy.visualization import simple_norm, ImageNormalize, ZScaleInterval, AsinhStretch` for image stretches

**corner** (`import corner`): corner plots of posterior samples. Docs

- `corner.corner(samples, labels=[...], levels=(0.68, 0.95), show_titles=True, truths=[...])`

**seaborn** (`import seaborn as sns`): statistical plots on top of matplotlib. Docs

- `sns.kdeplot(data=df, x="x", hue="group")`, `sns.histplot`, `sns.heatmap`
- `sns.color_palette("colorblind")` for a palette
- Careful: `sns.set_theme()` changes rcParams globally and will fight your style file.

**cycler** (`from cycler import cycler`): style cycles, for example `cycler(color=[...]) + cycler(linestyle=[...])` for `axes.prop_cycle`.

**Optional palette libraries** (only if installed): `cmasher`, `cmocean`, `colorcet`, `cmcrameri`. Each registers extra colormaps you use by name or as `cmap=cmr.ember`.

**Notebook magics:** `%%writefile name.ext` writes the cell's contents to a file (used for the style file).

## B. Line, marker and colour options

**Linestyles** (`ls=` or `linestyle=`; reference)

| Code | Name |
|---|---|
| `"-"` | solid |
| `"--"` | dashed |
| `":"` | dotted |
| `"-."` | dash-dot |
| `(0, (5, 1))` | custom: dash length 5, gap 1 |

**Markers** (`marker=`; reference)

| Code | Shape | Code | Shape |
|---|---|---|---|
| `"o"` | circle | `"s"` | square |
| `"^"` `"v"` `"<"` `">"` | triangles | `"D"` | diamond |
| `"*"` | star | `"P"` | filled plus |
| `"+"` | plus | `"X"` | filled cross |
| `"x"` | cross | `"."` | small point |

**Other line and marker options:** `lw` (line width), `ms` (marker size), `mfc` and `mec` (marker face and edge colour), `fillstyle="none"` (open markers), `markevery=5` (a marker every 5th point), `alpha` (transparency), `zorder` (drawing order), `drawstyle="steps-mid"` (stepped line), and for `errorbar`: `capsize`, `elinewidth`, `fmt`.

**Colours:** a name (`"tab:blue"`), a cycle colour (`"C0"` to `"C9"`), hex (`"#0072B2"`), a grayscale string (`"0.5"`), or an RGB tuple.

**Hatches** (`hatch=`; reference): `"/"`, `"\\"`, `"-"`, `"+"`, `"x"`, `"o"`, `"."`, `"*"` (and the vertical-bar pattern). Repeat a character for denser lines (`"///"`). Pair with `facecolor="none"` and `edgecolor=...`; `hatch.linewidth` controls the thickness.

## C. Colormap options

| Kind | Examples | Use for |
|---|---|---|
| Sequential | `viridis`, `plasma`, `inferno`, `magma`, `cividis` | Ordered, continuous data |
| Single-hue sequential | `Blues`, `Greys`, `Reds` | Light-to-dark emphasis |
| Diverging | `RdBu_r`, `coolwarm`, `PuOr`, `seismic` | Signed data with a meaningful zero |
| Cyclic | `twilight`, `hsv` | Angles and phases |
| Qualitative | `tab10`, `tab20`, `Set1`, `Set2`, `Dark2`, `Paired` | Separate groups |

Add `_r` to reverse any of them (`viridis_r`). `plt.colormaps()` lists them all. Avoid `jet` and `rainbow`.

**Colour scaling for images** (`imshow`, `pcolormesh`): `vmin` and `vmax`, or `norm=` with `LogNorm`, `SymLogNorm`, `PowerNorm`, `CenteredNorm` (symmetric about zero) or `TwoSlopeNorm` (a chosen centre), all from `matplotlib.colors`. Also `origin="lower"`, `extent=[...]`, `aspect="auto"`, `interpolation="nearest"`.

## D. Layout, ticks and legends

- **Figure and layout:** `plt.subplots(nrows, ncols, figsize=(w, h), sharex=True, constrained_layout=True)`; unequal panels, such as a plot with a residual panel: `gridspec_kw={"height_ratios": [3, 1]}`.
- **Scales:** `ax.set_xscale("log")`, `"symlog"` or `"linear"`; shortcuts `ax.loglog` and `ax.semilogy`.
- **Ticks:** `ax.tick_params(direction="in", top=True, right=True, which="both")`, `ax.minorticks_on()`.

- **Legend:** `ax.legend(loc="upper right", ncol=2, frameon=False, fontsize=9, bbox_to_anchor=(1, 1))`. `bbox_to_anchor` moves it outside the axes.
- **Text:** `fontsize`, `fontweight`, `ha` and `va` (alignment), `rotation`, `color`.
- **Saving:** `fig.savefig("fig.pdf", dpi=300, bbox_inches="tight", transparent=False)`; `rasterized=True` on a heavy artist keeps a PDF small.

---

## 🙏 Data credits and references

🔭

- **Haslam 408 MHz map:** Haslam, C. G. T. et al., 1982, A&AS, 47, 1; the destriped, source-subtracted version is Remazeilles, M. et al., 2015, MNRAS, 451, 4311 (arXiv:1411.3628), distributed by LAMBDA (used here at NSIDE 256).
- **Horsehead Nebula image:** a UK Schmidt photographic plate, as distributed with the Astropy documentation data ( `HorseHead.fits` ).
- **Gibbs-sampler chains** (convergence demo): real output from a Gibbs sampler (12 complex parameters, thinned by 10); the real parts of the first four are plotted.
- **H0 values:** the heatmap values are synthetic (drawn with a fixed seed), and the forest-plot values are approximate. Both are for illustration only, not real measurements.
- Colour and perception references: see sections 2 and 4.1.